

```

cardStack.push("ace of hearts");
document.write(cardStack + "<br />");
❸ cardStack.pop()
document.write(cardStack + "<br />");
</script>

```

When you run this code, the output looks like this:

```

5 of diamonds
5 of diamonds,3 of clubs
5 of diamonds,3 of clubs,ace of hearts
5 of diamonds,3 of clubs

```

The stack starts off as empty ❶. Three strings representing cards are pushed onto the stack ❷. Then the stack is popped ❸, which removes the ace of hearts and leaves the three of clubs at the top of the stack again. The state of the cardStack stack is tracked in Figure 1-6, going from left to right.

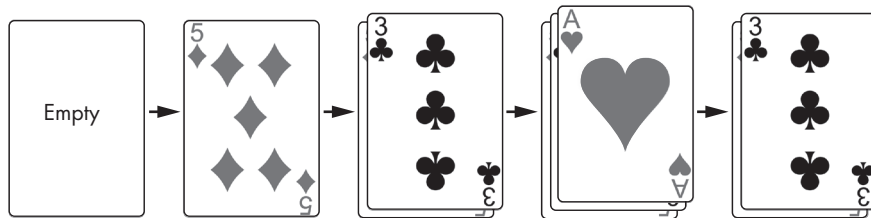


Figure 1-6: The stack starts empty. Cards are then pushed onto and popped off the stack.

You can see only the topmost card in the card stack, or, in our program’s stacks, the topmost value. In the simplest stack implementations, you can’t see how many cards (or values) are in the stack. You can see only whether the stack is empty or not.

Stacks are a *LIFO* data structure, which stands for *last in, first out*, since the last value pushed onto the stack is the first value popped out of it. This behavior is similar to your web browser’s Back button. Your browser tab’s history functions like a stack that contains all the pages you’ve visited in the order that you visited them. The browser is always displaying the web page at the “top” of the history’s “stack.” Clicking a link pushes a new web page onto the history stack, while clicking the Back button pops the top web page off and reveals the one “underneath.”

What Is the Call Stack?

Programs use stacks too. The program’s *call stack*, also simply called *the stack*, is a stack of frame objects. *Frame objects*, also simply called *frames*, contain information about a single function call, including which line of code called the function, so the execution can move back there when the function returns.